

Experiment 06

Name	: Johnson Kumar	UID	: 23BCS12654
Branch	: CSE	Section	: KRG-1A
Semester	: Sixth	Date of exp.	: 24/02/26
Subject	: System Design	Subject Code	: 23CSH-314

AIM

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

OBJECTIVE:

- To understand the architecture of a massive-scale media streaming system.
- To identify functional requirements such as direct-to-cloud uploads and personalized feeds.
- To identify non-functional requirements including CDN caching, high throughput, and latency constraints.
- To analyze the decoupling of synchronous API requests using event-driven stream processing.
- To design REST APIs for video management, search, and user interactions.

REQUIREMENTS

Functional Requirements:

- The system shall provide functionality for users to register, log in, and manage their accounts securely.
- The system shall support the efficient upload of large raw video files, ensuring reliability and scalability.
- The system shall enable smooth video playback through Adaptive Bitrate Streaming, dynamically adjusting quality based on network conditions.
- The system shall facilitate real-time social interactions, including likes, dislikes, and comments on videos.
- The system shall allow users to search for videos using titles or metadata attributes.

- The system shall generate a personalized home page feed tailored to each user's watch history and preferences.

Non-Functional requirements:

- The system must be capable of handling extremely high throughput, supporting concurrent uploads and millions of simultaneous video views.
- The system must ensure High Availability, eliminating any single point of failure.
- The system should adopt an Eventual Consistency model for social metrics (e.g., view counts, likes) to optimize database write performance.
- The system must minimize video delivery latency through the use of Edge Caching and a Global CDN.
- The transcoding pipeline must be horizontally scalable to efficiently manage spikes in viral uploads.

HIGH LEVEL DESIGN

Core-Entities

- Users
- Videos
- Likes/Dislike
- Comments
- History/other Playlist

Attached ott.drawio file has whole high-level design.

LOW LEVEL DESIGN

API Design

1. Authentication API

- A. Register: POST /api/v1/auth/register
- B. Login: POST /api/v1/auth/login

2. Video & Feed API

- A. Initialize Upload:
POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
- B. Get Video Metadata: GET /api/v1/videos/{video_id}
- C. Get Personalized Feed: GET /api/v1/feed (Pagination Supported)

D. Search Videos:

GET /api/v1/search?q={query}

3. Interaction API

A. Like Video: POST /api/v1/videos/{video_id}/like

B. Add Comment: POST /api/v1/videos/{video_id}/comments

C. Get Comments: GET /api/v1/videos/{video_id}/comments

LEARNING OUTCOMES

- Understood the overall architecture of a large-scale video streaming and viewing platform, including client applications, backend services, and CDN integration.
- Identified the functional requirements such as user registration, video uploads, adaptive bitrate streaming, personalized feeds, and social interactions.
- Identified the non-functional requirements including scalability, high throughput, low latency, high availability, and eventual consistency for social metrics.
- Analyzed how synchronous API requests could be decoupled using event-driven stream processing for scalability and resilience.
- Designed REST APIs for video management, search, and user interactions that support modularity and extensibility.
- Evaluated the role of CDN caching, transcoding pipelines, and edge delivery in minimizing latency and ensuring smooth playback.
- Applied principles of horizontal scalability and fault tolerance in handling viral content spikes and millions of concurrent viewers.
- Created a structured HLD document that captured system components, data flows, and integration points for a streaming platform.